Overall Feedback

While each arrow has its corresponding relationship symbol, however adding the actual relationship helps to clear up any formatting differences. This can be applicable to arrows that do not use the default UML format and helps with the overall readability of the diagram, especially when multiple classes are interacting with one another.

Additionally, it is useful to both the reader and the programmer to see the input arguments and return types for each method/constructor. When multiple classes are interacting with one another, it should show how these classes relate. For example, if object A collects object B (aggregation) then it must show object B in the field. This improves readability as well as show the boundaries of the relationship.

Additional specific feedback is indicated in blue container below.

Suggestion: The game class uses pieces as objects, and will be destroyed when game object ends. To better capture this relationship a stronger from of association is better suited, possibly composition or aggregation.

Feedback: Since there are various different games that require the same features such as start game, take turn, and reset game, Game class should be abstract. By making the class abstract, other games can extend the class and tailor to their personal logic while keeping the functionality the same.

Feedback: Unclear how types of pieces are assigned, it would be ideal to use piece as an abstract class where there can be class that extend this class for each game. Additionally, enumeration can be used here where the piece constructor takes in an enum and sets its type.

Suggestion: checkWinCondition() and checkTieCondition() can be moved to GameLogic class where it is more appropriate. Rather than each game having different WinCondition(), it is better suit to have extensions of GameLogic since game for the most part are similar, however logic is different.

Feedback: Unclear how multiple players will interact with the board, the diagram only shows a single player interaction. May use queues along with sequence assigning to ensure each player gets their turn. Ideally this would have one to many relationship with the game since multiple players are active during game play.

Suggestion: The game class uses pieces as objects, and will be destroyed when game object ends. To better capture this relationship a stronger from of association is better suited, possibly composition or aggregation.

Feedback: Composition with multiplicity is not ideal here since user can have one curated profile. Having multiple profiles does not enhance the functionality of the application and can instead create new logins. In this case the multiplicity should be removed to have 1...1 relationship.

Suggestion: Display methods should be part of the GUI rather than the back end portion of the profile. However, methods such as displayStats() can return a string of data that can be passed to the GUI to be printed, which would require the method to have a return type of string.

Feedback: There is conflicting information regarding the rank, player class on the right it shows that rank is a enum while here it shows as a string. Having consistency across the diagram is key to ensure all classes interact with other classes in the same manner.

Suggestion: EmailService should be connected to the networking class to ensure that the class is utilized appropriately. Without having any network communication it is difficult to transmit data for confirmation and resetting password.

Suggestion: EmailService will require authentication to prove the email exists, and/or any other proof of identity to ensure the user is authorized to change credential. This can either be integrated within the class using helper methods, or can use AuthenticationService in which case relationship be reversed.

Suggestion: viewCurrentProfile() can be connected to user stats that shows all the details. Rather than having a method that performs similar functions as a class it is beneficial to use the UserStats with an association.

Suggestion: CurrentStatus can be changed into a enum which will be set when creating the profile or toggled by the user. Having a string type is not ideal when setting the status for accessiblity and may require additional error checking.

Suggestion: Rank is set in the profile, rather than having a field with the same attribute, it is best to reduce redundancy. Can also implement a method in user that can get all field attributes to set userstats.

Suggestion: GameHistory is stored into an arraylist, which can be better captured using association with multiplicity. This shows that these objects will tracked simultaneously.

Feedback: AuthenticationHandler should have an association with User to ensure that when user creates profile, the login data is prepared and authenticated. Ideally, service handles this property which is counter intuitive since Handler prepares login data.

Feedback: ClientConnection class does not appear on the class structure diagram including the documentation for java standard libraries. To ensure readability and clarity, ensure all classes are documented on the diagram.

Comment: Java.net, java.NIO, and java.lang.Thread are java standard libraries that used in the project. The representation of these libraries is shown appropriately and clearly indicates that these libraries will be associated with the Networking class.

Feedback: LeaderBoardEntry is not a class represented on this class structure diagram. It is unclear how this class interacts and what corresponding details this class contains. Ensure all classes that are used are shown on the diagram.

Suggestion: Leaderboard can benefit from an rank comparator class that will decide the sequence that the players are ordered. The comparator method is useful to add complex behaviour that may apply to the rank, such as skill based match making rank or game pieces moved to win. These attributes can determine the leaderboard in greater detail to show how one player stacks up to another. Furthermore, it can be coupled with the "equal" method under the player class to ensure consistency.

Feedback: Remove association arrow. Association here is unnecessary since composition is a stronger association relationship therefore is implicitly denoted.

Feedback: Networking class acts as the hub to connect and control all of the components within the project. This way the networking class owns ChatManager, GameSyncManager, and AuthenticationHandler using an composition relationship. In this case all of the composition arrows must be reversed to ensure the correct relationship is communicated.

Feedback: BoardState is missing from the class structure diagram. To ensure that relationships and interactions are clear, documentation for these classes is necessary.

Suggestion: Matchmake should have many to one relationship with player to show that multiple player objects are being collected when interacting with the queue. This relationship aligns with the UML format to ensure readability and show the boundaries of the relationship.

Suggestion: Unless the placement object is recreated for each instance the player is in matchmaking it would be ideal to add reset placement test to remove previous results. This way the object can be reused and be optimized better for the application.

Suggestion: Player should have an association relationship with multiplicity since leader board tracks multiple player objects and are stored in a custom list. This relationship makes it to the programmer and reader that all methods of the player class are accessible when updating the leaderboard in their respective methods.

Suggestion: Composition arrow should be reversed, player class owns rank objects. This way when player object is destroyed the rank (enumeration) object is destroyed alongside.

Suggestion: Unless this instance of the player class is under its own package, there will be a naming conflict with a player class that interacts with the game board. Ensure class naming is handled appropriately to eliminate any conflicts.

Suggestion: UpdatePlayerTTT, updatePlayerCheckers, updatePlayerC4 should be moved in to their respective classes rather than in the super class. This ensures that each instance class does not have to implement methods that are irrelevant to those classes.

Suggestion: LeaderBoard should have an abstract label that displays that the class cannot be instantiated and needs to be extended through personalized classes (as shown below).

Suggestion: Ideally enumeration classes are used to set status by a predefined value. These ranks should be upper caps to align with format of constants. Additionally, these ranks can have predefined value assignment for easier tracking and comparison.

Suggestion: As discussed in the game logic section, ideally these games can extend the game class, given game class is an abstract class. This would allow for similar functionality while modifying the game logic individually.

Feedback: ChatManager is a superclass that tracks and manages the Chatbox GUI, therefore the association arrow should be reversed. This ensures that ChatManager handles the requests sent by the Chatbox such as displayMessages.

Suggestion: AnimatedLoginScreen should be associated with the networking class since the data from the text fields is sent to be authenticated. Without having this relationship, the AuthenticationHandler and Service cannot interact, therefore should be appropriately handled to ensure a seamless transfer of data.

Suggestion: AnimatedLoginScreen should account for other components that are used such as alerts that can be displayed for error handling and listener objects that that track user inputs.

Suggestion: All of the components that are dependencies of MainGamePlatform require the JavaFX library import. This diagram will benefit from showing this library, since most GUI is based around these library components.

Feedback: Arrows should be moved towards the centre of the class container. With this current layout, it is unclear where each of these arrows are leading to and from. This reduces readability and must make assumptions about the diagram.

Feedback: Association arrow should be reversed, LobbyPage is the superclass and ChatBoxGUI will be apart of the lobby as an additional function.

Feedback: Unclear how different variations of the game will be displayed. PlayGame can be an abstract class which the individual games can have classes and override the methods defined in PlayGame.

Suggestion: The JavaFX program can benefit from a debugging/console log output that records any errors or success messages. This ensures that user and programmer can identify the issue that may be caused and not caught by other methods. It can work well with the MainGamePlatform which is the main JavaFX executable code, since launching may cause crashes that are difficult to debug. This output file can be created everytime the application is launched and/or when the user executes a specific step, and previous file can be overwritten.

Piece
- type
- color
+ getType()
+ getColor()
+ promote()

Game
- currentPlayer
- gameState
+ startGame()
+ takeTurn()
+ checkWinCondition()
+ checkTieCondition()
+ resetGame()

0..*

1

Player
- symbol
+ makeMove(position)

Board
- grid
+ initialBoard()
+ getPieceAt (position)
+ setPieceAt(position)
+ setPieceAt(position, piece)
+ isFull()

GameLogic
+ checkVictory()
+ checkDraw()
+ switchPlayer()

EmailService
+ sendConfirmationEmail(email: String): boolean
+ sendPasswordResetEmail(email: String): boolean
+ sendDeletedProfileEmail(email: String): boolean

AuthenticationService
+ validateCredentials(username: String, password: String): boolean
+ resetPassword(email: String): boolean

User
- userID: int
- username: String
- password: String
- email: String
- profile: Profile
- rank: String
+ login(username: String, password String): boolean
+ logout(): void
+ registerNewUser(userName: String, password: String, email: String): void
+ updateUsername(newUsername: String): void
+ updateEmail(newEmail: String): void
+ viewCurrentProfile(): Profile

1..*

Profile
- userID: int
- username: String
- gamesPlayed: ArrayList<GameHistory>
- userStats: UserStats
- currentStatus: String
- record: Record
+ displayStats(): void
+ viewGameHistory(): void
+ updateStats(newStats: UserStats, newGames: GameHistory: newRecord: Record): void
+ searchProfile(username: String): Profile

Record
- totalWins: int
- totalLosses: int
- totalDraws: int
+ getRecordDetails(): void
+ updateGameRecord(): void

UserStats
- totalGamesPlayed: int
- wins: int
- losses: int
- draws: int
- rank: String
+ getRankDetails(): void
+ updateRank(newRank: String): void
+ calculateWinRatio(): double

GameHistory
- gameID: int
- gameName: String
- datePlayed: int
- opponent: String
- score: int
- outcome: String
+ displayGameHistory(): void

Matchmake
- playerQueue: Queue<Player>
+ addPlayerToQueue(player: Player): void
+ removePlayerFromQueue(player: Player): void
+ returnBestMatch(playerQueue: Queue<Player>): ArrayList<Player>
+ canMatchMake(playerQueue: Queue<Player>): boolean
+ clearQueue(): void

PlacementTest
- round1: int
- round2: int
- round3: int
- round4: int
- round5: int
+ PlacementTest (player: Player)
+ setScore (round: int, score: int) :void
+ setRank (gameplayed: int): void

Player
- username: String
- id: int
+ placementGamesLeft: int
- rank: enum Rank
+ equals(obj: Object): boolean

0..*

1

<<enumeration>> Rank
Iron
Bronze
Silver
Gold
Platinum
Diamond
Immortal
Radiant

LeaderBoard
players: List <Player>
+ addPlayer(player: Player): void
+ updatePlayerC4(player: Player, won: boolean): void
+ updatePlayerTTT(player: Player, won: boolean): void
+ updatePlayerCheckers(player: Player, won: boolean): void
+ recalculateMMR (player: Player, game: int): void
+ displayLeaderBoard(): void

AdjustRating
- minimum_mmr_gain: int
- minimum_mmr_loss: int
- mmrChange: int
+ updateRating(): void
+ updateRank(): void

CheckersLB
+ displayLeaderBoard(): void

TicTacToeLB
+ displayLeaderBoard(): void

ConnectFourLB
+ displayLeaderBoard(): void

+ AuthenticationHandler
- username: String
- password: String
+ prepareLoginData(username: String, password: String)
+ prepareRegData(username: String, password: String)
+ receiveAuthResponse(response: String)
+ checkAuthentication(): boolean

+ ServerRelay
- connectedClients: List<ClientConnection>
- matchQueue: Queue<ClientConnection>
+ authenticateClient(client: ClientConnection): boolean
+ matchClients()
+ relayData(sender: ClientConnection, receiver: ClientConnection, data: String)

+ LeaderboardHandler
- leaderboardData: List<LeaderboardEntry>
+ requestLeaderboard()
+ updateLeaderboard(data: List<LeaderboardEntry>)
+ saveLeaderboardData()

+ Networking
- msgToBeSent: String
- connectionStatus: Boolean
+ startComponents()
+ runLoop()
+ requestGame(gameType: String)
+ sendData(data: String)
+ receiveData(): String

Java.net

java.NIO

java.lang.Thread

+ ChatManager
- messageQueue(): Queue<Message>
- msgToBeSent(): Message
+ sendMessage(msg: Message)
+ receiveMessage(): Message
- decodeMessage(msg: Message)

+ GameSyncManager
- boardState: BoardState
+ sendBoardState(boardstate: BoardState)
+ receiveBoardState(): BoardState
+ endUserTurn()
+ setUserTurn(playerID: int)

+ sendBoardState(boardstate: BoardState)
+ receiveBoardState(): BoardState
+ endUserTurn()
+ setUserTurn(playerID: int)

For sound effect

+ Media
- audioFiles: List<String>
+ playsound(): String
+ soundTransition()
+ stopSound(): Boolean
+ soundSpeed() : int

+ AnimatedRegistrationScreen
- registration: Form
- storage: Query
+ displayRegistration()
+ displayRegistrationMessage()
+ lobbyTransition()

+ AnimatedLoginScreen
- usernameTextbox: TextField
- password Textbox: TextField
- backgroundPicture : String
+ displayLoginScreen()
+ authentication(String, String)
+ displayLoginMessage()

+ MainGamePlatform
- availableGames: List<String>
+ displayGames()
+ chooseGame()
+ choosePlayers()
+ notifications()

+ Chatbox GUI
- currentPlayers() : Player
+ displayMessages()
+ displayChatbox()

+ PlayGame
- board: GridPane
+ displayInstructions()
+ displayGameBoard()
+ displayLeaderBoard()
+ displayWinner()

+ LobbyPage
- currentPlayers() : Player
+ displayStatusMessage(String): String

Suggestion: As UML diagram format, the method frame should identify an input arguments within the label. This would make it clear to when programming the required inputs as shown in the first dispatch arrow.

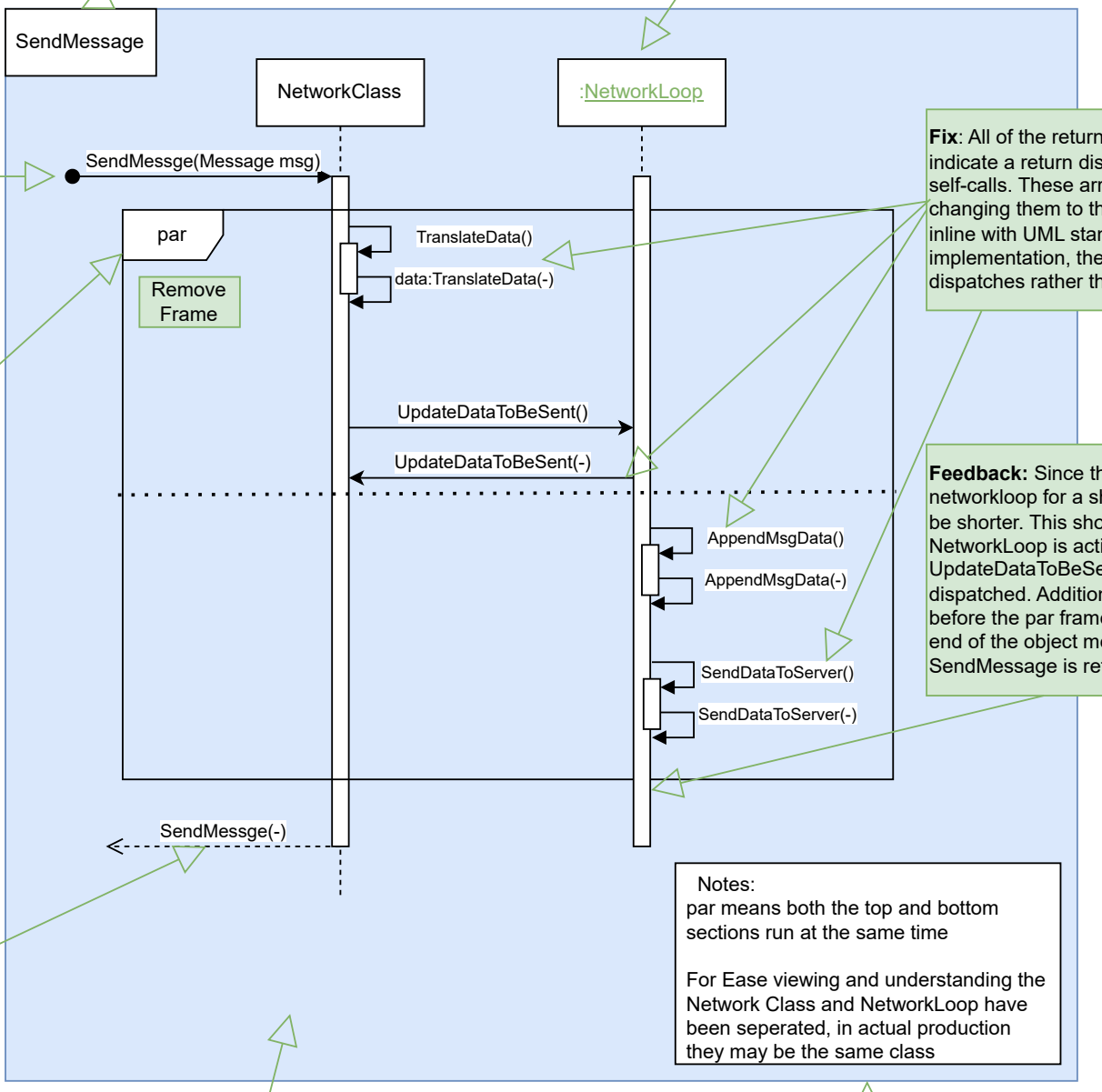
Feedback: Unclear if this life line represents an object or role. Typically using :Obj represents an object of the NetworkLoop which can be further clarified by underlining.

Suggestion: Including an actor in this sequence diagram would be an addition that needs to be added for the following reasons:

1. Adding actor increases the readability, helping programmers and production team to identify the user that interacts with this diagram. These indicate the limits of the diagram and which user is able to access the said methods.

2. Indicate the dependencies the diagram requires, and which users can execute the flow of the sequence. Without this it would be unclear when and how the described method will be dispatched.

3. Connecting the sequence diagram with its corresponding use case actor would enable a seamless flow of details. Readers can read use case diagrams and seek sequence diagrams for greater insight.



Fix: All of the return arrows must be dashed to indicate a return dispatched is being sent, including self-calls. These arrows need to be fixed by changing them to their appropriate format to be inline with UML standards. Without this implementation, the return arrows are shown as dispatches rather than their intended purpose.

Feedback: While the par frame is useful for showing processes that run parallel to one another, here it is not used as intended. The UpdateDataToBeSent() is the trigger for AppendMsgData() and SendDatatoServer() which require the data to be sent first. Therefore here it would better to remove a par frame since these processes are parallel rather require data to be sent in sequence.

Feedback: Since the NetworkClass depends on the networkloop for a short duration, the life line must be shorter. This should be done to indicate that the NetworkLoop is active for the duration of the UpdateDataToBeSent() not before the message is dispatched. Additionally, the NetworkLoop must end before the par frame is terminated to indicate the end of the object methods, and that the SendMessage is returned.

Suggestion: It is unclear the return type of SendMessage, and it would be difficult to understand the context of SendMessage without having a return type.

Possible Return Types:

1. Boolean: This would indicate whether the message has been sent successful, possibly being used as a flag to throw exceptions.

2. **Void:** This would indicate the message completes the required processes without indicating to user, and where system interrupts are done through exceptions.

3. **Enum:** Can be used similar to other class elements in the structure diagram which sets a enum to the message, and returns the enum as a message. This enum can be used to handle exceptions and indicate to user the specific error.

Comment: Background colour is a good addition to the diagram as it helps to define and put emphasis on key details of the diagram.

Suggestion: Place this in the top right corner before the user begins reading. This helps with the flow of the diagram since the user is equipped with the information before beginning to read.

Comment: This notes section is useful for reading the diagram, traditionally the par (standing for *parallel*) is not used and serves to inform the readers. Description is a great addition to the diagram to indicate a short summary of the information.

GAME LOGIC USE CASE DESCRIPTIONS:

CONNECT 4 USE CASE DESCRIPTIONS:

Start Game:

- Primary Actor(s): Player 1, Player 2
- Goal in context: to start a new Connect 4 game with an empty board and ready for players to take turns
- Trigger: players choose the option to start a new game
- Preconditions:
  - the game application is running
  - no active game is in progress
- Scenario:
  - players open the connect 4 application
  - players select the option to start a new game
  - the system initializes an empty 7x6 board
  - the system assigns player 1 as Red and player 2 as Yellow
  - the game indicates that player 1's turn is first by a label "Red's turn"
- Post-conditions:
  - the game board is displayed with all slots empty
  - the turn indicator at the bottom left shows that it is player 1's (RED's) turn
- Exceptions: if the "New Game" button is pressed during an active game, the system displays a message asking players to confirm
- Priority: High
- Channel to actor: GUI; game window with the "New Game" button
- Secondary actors: system
- Channel to secondary actors: internal game logic, resetting board and player turns
- Open issues: none

Take Turn (Drop Piece):

- Primary Actor(s): Player 1, Player 2
- Goal in context: allows the current player to drop their piece into a selected column
- Trigger: the current player clicks on a column in the game window
- Preconditions:
  - a game is in progress
  - it is the current player's turn indicated by the turn label in the bottom left
- Scenario:
  - the system displays the current player's turn (red or yellow)
  - the player selects a column by clicking on it
  - the system checks if the selected column has an empty slot
  - the system drops the player;s piece in the lowest available slot in that column
  - the system updates the board display
  - the system checks for win or tie conditions

    - if there is no win or tie, the system switches to the next player and updates the turn label
- Post-conditions:
    - the selected column has 1 additional piece in the lowest empty slot
    - the turn alternates to the next player if there is no tie or win
- Exceptions: if the selected column is full, the system alerts the player to choose another column
- Priority: high
- Channel to actor: GUI; game window where players click columns
- Secondary actors: system
- Channel to secondary actors: internal game logic thats updating the board and checking conditions
- Open issues: none

Check Win Condition:

- Primary Actor(s): system
- Goal in context: determines if the last move resulted in a win by forming a line of four consecutive pieces
- Trigger: a player has just successfully placed a piece in a column
- Preconditions: a piece has been dropped in the game board
- Scenario:
    - the system identifies the piece's position
    - the system checks for 4 consecutive pieces of the same color in all directions: horizontal, vertical, and diagonal (both directions)
- Post-conditions:
    - if a winning line is found, the game ends and the winner is announced
    - if no winning line is found, the game proceeds to the next turn
- Exceptions: none
- Priority: high
- Channel to actor: system
- Secondary actors: none
- Channel to secondary actors: none
- Open issues: none

Check Tie Condition:

- Primary Actor(s): system
- Goal in context: determines if the board is full without a winning line, resulting in a tie
- Trigger: a player has just successfully placed a piece, and no win condition is met
- Preconditions:
    - a piece has just been placed
    - no winning line exists
- Scenario:
    - the system checks if all columns are full
    - if the board is full and no winning line exists, the system declares the game a tie

- Post-conditions:
    - if the board is full and no win is detected, the game ends, and the system announces a tie
    - if there is still space on the board, the game continues
- Exceptions: none
- Priority: medium
- Channel to actor: system
- Secondary actors: none
- Channel to secondary actors:none
- Open issues: none

Display Game Result:

- Primary Actor(s): system
- Goal in context: display the outcome of the game (win or tie) to the players
- Trigger: a win or tie condition is detected
- Preconditions: a win or tie has been detected by the system
- Scenario:
    - the system identifies the game's end due to a win or tie
    - if there is a win, the system shows a message that Player 1 (Red) or Player 2 (Yellow) has won
    - if there is a tie, the system shows a message stating "It's a tie!"
    - the system offers players the option to start a new game by pressing the "New Game" button
- Post-conditions:
    - the game result is displayed to the players
    - players are given the option to start a new game
- Exceptions: none
- Priority: medium
- Channel to actor: GUI
- Secondary actors: Player 1, Player 2
- Channel to secondary actors: GUI (pop-up messages)
- open issues: none

Reset Game:

- Primary Actor(s): Player 1, Player 2
- Goal in context: reset the game board and prepare for a new game after a win or tie
- Trigger: players press the "New Game" button after a game has ended
- Preconditions:
    - a game has ended due to a win or tie
    - players select "New Game"

- Scenario:
    - players press the "New Game" button
    - the system calls the resetGame() method

    - the system reinitializes an empty 7x6 board
    - the system resets the current player to Player 1 (Red)
    - the turn label is updated to show "Red's turn"
    - the board display is refreshed to show an empty grid
- Post-conditions:
    - the game board is cleared, and a new game is ready to begin
    - Player 1 (Red) is set to take the first turn

- Exceptions: none
- Priority: medium
- Channel to actor: GUI with the "New Game" button
- Secondary actors: system
- Channel to secondary actors: internal game logic with resetting board and player turnd
- Open issues: None

Exit Game:

- Primary Actor(s): Player 1, Player 2
- Goal in context: end the game session and close the game application
- Trigger: players close the game window or select "Exit"
- Preconditions: the game window is open
- Scenario:
    - players choose to close the game window or select an exit option
    - the system checks if an active game is in progress
    - if a game is in progress, the system prompts players to confirm exiting
    - upon confirmation, the game window closes, and the application terminates
- Post-conditions: the game window is closed and the application is terminated
- Exceptions: if players choose not to exit and the game window remains open
- Priority: low
- Channel to actor: GUI with the window close button
- Secondary actors: none
- Channel to secondary actors: none
- Open issues: none
-

TIC-TAC-TOE USE CASE DESCRIPTIONS:

Start Game:

- Primary Actor(s): Player
- Goal in context: The system initializes the game board and display it to the active player.
- Trigger: The player requests that a new game start.
- Preconditions: None

- Scenario:
    - System initializes an empty 3x3 game board
    - System makes active player 'X'
    - System displays the game board to the active player
- Postconditions: The game board is displayed and active player set as 'X'.
- Exceptions: None

Make Move:

- Use Case: Make Move
- Primary Actor(s): Player
- Goal in Context: The player can highlight a cell on the board.
- Trigger: It is the active player's turn .
- Preconditions: The board is displayed and there is an active player.
- Scenario:
    - The player selects a cell by inputting the coordinates of that cell.
    - System checks to see if the cell is in the bounds and not occupied.
    - If it is, then the system marks that cell as taken by the active player with their symbol.
- Postconditions: The chosen cell is highlighted with the player's mark.
- Exceptions: Input cannot be a valid move or the cell is taken

Check Victory:

- Primary Actor(s): System
- Goal in context: To check if active player has won
- Trigger: A player performs a valid move
- Preconditions: A valid move has been performed
- Scenario:
    - System checks the row, column and diagonal for three consecutive symbols
    - Upon successful completion of finding a valid winning condition, it declares the active player as the winner
- Post Conditions: Game announces the active player as winner in case of the above conditions
- Exceptions: None

Check Draw:

- Primary Actor(s): System
- Goal in context: Determine if game is a draw.
- Trigger: All cells in the game board have been filled.
- Preconditions: The cells are filled, and there is no reached winning condition.
- Scenario:
    - System checks the board to confirm all cells are set.
    - If there isn't a winning condition, state that this is a draw.
- Postconditions: Game states a draw.
- Exceptions: None

Switch Player

- Primary Actor(s): System
- Goal in context:: The active player should switch between 'X' and 'O'.
- Trigger: A valid move was made, and no victory or draw condition was reached.
- Preconditions: The last move was valid and didn't lead to the end of the game.
- Scenario:
  - The system switches the active player to the next player ('X' or 'O').
- Post-Conditions: The active player is switched.
- Exceptions: None

Play Again:

- Primary Actor(s): Player
- Goal in context: The system will ask the player if they want to play again or quit.
- Trigger: The game ends with either a win or a tie.
- Preconditions: The game has been won or tied.
- Scenario:
  - The system prompts the player if they would like to play again.
  - If the player chose 'yes', the system should restart and the game board should reset.
  - If the player chose 'no', the system should exit the game.
- Postconditions: Game restarts in case of 'yes' else exits.
- Exceptions: Player enters an invalid input

CHECKERS USE CASE DESCRIPTIONS:

Start Game:

- Primary Actor(s): Player 1, Player 2
- Goal in context: To initialize all pieces in the standard starting position to begin a game.
- Trigger: Players choose to start a new game.
- Preconditions:
  - The players have launched the Checkers application.
- Scenario:
  - The players starts a new game.
  - The system creates an 8x8 board with all the pieces placed for each side in their respective squares.
- Post-conditions:
  - Pieces are all in place and the game is ready to start.
  - Player 1 is prompted to make the first move.
- Exceptions:

    - Incorrect Setup: If the pieces are not arranged correctly, the system resets the board to the initial configuration.
- Priority: High. If the board is not set the game cannot begin.
- Channel to actor: via clicking with a mouse on the start game button
- Secondary actors: System
- Channel to secondary actors: Notified that a player wants to start a game, and sets up the board and prompts a player to make their move.
- Open issues: N/A

Move Piece

- Primary Actor(s): Current player (Player 1 or Player 2)
- Goal in context: To move a piece on the board.
- Trigger: Player selects a piece from their respective side and an available square to move it to.
- Preconditions:
    - It is currently the player's turn
    - Player has selected a piece from their respective side that they would like to move
    - Player has selected an available square they want to move the selected piece to
- Scenario:
    - It becomes the player's turn
    - Player selects a piece they want to move
    - GUI displays the spaces the piece can move to (diagonal, away from the player's side of the board)
    - Player selects an open square to move the piece to
    - Piece is moved to the selected square
- Post-conditions: Piece has been moved to the player's desired square
- Exceptions:
    - There are no open squares for the selected piece to move to
    - The square that the player wants to move the piece to already has another piece on it
- Priority: High, core function in the game
- Channel to actor: via clicking with a mouse on the GUI screen
- Secondary actors: System
- Channel to secondary actors: Shows player where the selected piece can move, then moves the piece once player has made their selection
- Open issues: N/A

Move King:

- Primary Actor(s): Current player (Player 1 or Player 2)
- Goal in context: To move a king on the board

- Trigger: Player selects a king from their respective side and an available square to move it to.
- Preconditions:
    - It is currently the player's turn
    - Player has selected a king from their respective side that they would like to move
    - Player has selected an available square they want to move the selected king to
- Scenario:
    - It becomes the player's turn
    - Player selects a king they want to move
    - GUI displays the spaces the piece can move to (diagonal, either towards or away from the player's side of the board)
    - Player selects an open square to move the king to
    - King is moved to the selected square
- Post-conditions: King has been moved to the player's desired square
- Exceptions:
    - There are no open squares for the selected piece to move to
    - The square that the player wants to move the piece to already has another piece on it
- Priority: High, core function in the game
- Channel to actor: via clicking with a mouse on the GUI screen
- Secondary actors: System
- Channel to secondary actors: Shows player where the selected king can move, then moves the king once player has made their selection
- Open issues: N/A

Capture Opponent Piece:

- Primary Actor(s): Current player (Player 1 or Player 2)
- Goal in context: To capture an opponent’s piece by jumping over it diagonally.
- Trigger: The current player selects a piece to make a capture move.
- Preconditions: An opponent’s piece is diagonally adjacent, and the square beyond it is empty.
- Scenario:
    - The player selects a piece to jump over an opponent’s piece.
    - The system verifies the move is a valid capture.
    - The player’s piece moves over the opponent’s piece to the next diagonal square.
    - The captured piece is removed from the board.
- Post-conditions:
    - The opponent’s captured piece is removed from the board.
    - The player’s piece is on the destination square.
- Exceptions:
    - Invalid Capture: If the capture move does not follow the rules, the system disallows the move.
- Priority: High, core function in the game
- Channel to actor: via clicking of mouse on the GUI screen

- Secondary actors: System
- Channel to secondary actors: Notified of the piece that has been jumped and updates the board accordingly
- Open issues: N/A

King Promotion:

- Primary Actor(s): Current player (Player 1 or Player 2)
- Goal in context: To promote a piece to a king when it reaches the opposite side of the board.
- Trigger: A standard piece reaches the opponent's back row.
- Preconditions: The moving piece is not a king and reaches the last row of the opponent's side.
- Scenario:
    - The player moves their piece to the opponent's back row.
    - The system recognizes that the piece has reached the end and promotes it to a king.
    - The piece gains the ability to move and capture both forward and backward.
- Post-conditions:
    - The piece is now a king and can move in both directions.
- Exceptions:
    - Missed Promotion: If a piece isn't promoted upon reaching the back row, the system automatically promotes it at the start of the next turn.
    - Priority: High, core function in the game
- Channel to actor: via clicking of mouse on the GUI screen
- Secondary actors: System
- Channel to secondary actors: notified that a piece has reached the other end of the player's board
- Open issues: N/A

End Game:

- Primary Actor(s): Player 1, Player 2
- Goal in context: To conclude the game by declaring a winner when one player either captures all opponent pieces or blocks all opponent moves, or to end the game as a draw if neither player can continue.
- Trigger: A victory condition is met (one player has no remaining pieces or moves).
- Preconditions: The game has progressed to the point where one player has either lost all pieces, is blocked from making any valid moves, or both players have no valid moves left
- Scenario:
    - The system checks for victory conditions at the end of each turn.
    - If one player has no remaining pieces or is blocked from making any valid moves, the system declares the other player as the winner.

    - If both players have no pieces or no possible moves, the system declares the game a draw.
- Post-conditions:
    - The game ends, displaying the winner (if one exists) or a message declaring a draw.
- Exceptions:
    - Draw Scenario: If both players are blocked from moving or have no pieces left, the game ends in a draw.
    - Forfeit Scenario: If a player forfeits the match or disconnects; the system declares the other player as the winner.
- Priority: High, core function in the game
- Channel to actor: will be notified the game has ended on the interface
- Secondary actors: System
- Channel to secondary actors: Checks conditions after each player's turn
- Open issues: N/A

NETWORKING USE CASE DESCRIPTIONS:

Send Message

- Primary Actor[s]: Clients
- Goal in context:
    - To allow for any Clients to communicate with each other via a textual Message
- Pre-Conditions:
    - Client is logged in
    - Client has verified their account
- Trigger:
    - Client would like to send a message to another Client
- Scenario:
    - Client selects account to send a message to
    - Client types message
    - Client sends message
    - Recipient Client receives message
    - Recipient Client sends message back

- Post-Conditions:
    - Messages are sent and received by the Clients
- Exceptions:
    - Account has been banned from communication for inappropriate behavior
    - Message not sent due to inappropriate language

    - Message not sent because Recipient has blocked Client
    - Message not sent due to too many messages sent in short succession
    - Message not sent due to Recipient disabling messages
- Priority:
    - High, this is one of the major features of the software
- When available:
    - First iteration

- Frequency of Use:
    - Many times daily
- Channel to actor:
    - Interaction with Client account
- Secondary actors:
    - Recipient Client
- Channel to Secondary Actor:
    - Message received by first client
- Open Issues:
    - None

MATCHMAKING AND LEADERBOARD USE CASE DESCRIPTIONS:

Matchmake

- Primary Actor(s): Player
- Goal in context: Determine the appropriate opponent to place against based on the player's rating
- Trigger: The player begins queueing into a game
- Preconditions: The player must have a valid account, and be eligible to play
- Scenario:
    - The player wants to begin a game, so they start a game
    - The system checks the player's profile statistics and ELO
    - The system searches for others players actively in queue, who share a similar skill level
    - The system finds a similarly skilled player to the player, and places them in the same game
- Postconditions: The player begins a game with a similarly skilled opponent
- Exceptions: The player is not eligible to play due to account restrictions or other ineligibilities.

Adjust Rating

- Primary Actor(s): Player
- Goal in context: To make accurate adjustments to a player's rating, or rank if the threshold is met, for each individual board game based on their performance in the most recently completed match. The direction and degree of this adjustment is determined

based on a variety of factors: If the player won or lost the match, the number of moves made during the match, the time required to complete the match, etc.

- Trigger: The player completes a match, triggering the adjustment to occur.
- Preconditions:
  - The player must have already completed their placement test that determines their rank for that specific game.
  - The player must complete the match in its entirety, resulting in either a win, loss, or draw.
- Scenario:
  - Player loads into a match of connect 4 against another player of similar rank.
  - The player wins the match in 4 moves
  - The system registers that a player has won and terminates the game
  - The player gets a gain in their rating for winning, and gets an additional gain in rating due to winning in a quick fashion.
  - The system checks if the minimum threshold for the next rank is met, and updates the player's rank if true.
  - Player's position on the leaderboard is updated.
- Secondary Actor: System
- Postconditions:
  - Player's rating and/or rank has been accurately updated.
- Exceptions:
  - One of the players unintentionally disconnects or forfeits the match before its completion.
  - A player cannot gain or lose rating if they are already at the maximum or minimum rank respectively.
  - A player loses a game due to their opponent cheating.

Placement Test

- Primary Actor(s): player
- Goal in context: To determine and add the player's starting leaderboard rank for a specific board game. This is accomplished by the player participating in a placement test for a specific type of board game. This placement test consists of five rounds of various difficulties to determine the player's ranking. After the placement test, the player's ranking will be updated to the leaderboard where they will then be able to compete with other players.
- Trigger: The platform detects that the player does not have a ranking for the specific board game they are trying to play. The player is asked to participate in the placement test for one of the three possible games they are trying to play.
- Preconditions:
  - A ranking does not exist for the game the player is trying to play.
  - The player has an account, this is needed to allow linking of the ranking to the associated player account.
- Scenario:

        - player joins the board game platform
        - The player selects one of the three games to play
        - The account is checked to see if the player already has a ranking
        - players without a ranking, will be promoted to initiate the placement test for this game
        - The placement test will be initiated
        - The player plays five rounds of the placement test where the score for each round is recorded
        - The score of each round is averaged out where this is used to determine the ranking of the player for the selected game
- Postcondition:
    - A player's ranking is added correctly to the leaderboard
- Exceptions:
    - The player already has a ranking for the game selected
    - The player does not have an account, therefore unable to link the ranking with the associated player
    - The player quits mid way completing the five rounds of the placement test, therefore, there will be no rank to update
    - The player does not wish to participate and does not initiate the placement test by leaving the page

Updating the Leaderboard

- Primary Actor(s): player
- Goal in context: Updates the leaderboard with the player's new score after completing a game.
- Trigger: The game session ends with a clear winner between the player and a matched opponent.
- Preconditions:
    - The player has successfully completed a game session.
    - The player has already completed their placement games, or has just completed their last placement game.
    - The leaderboard system is operational and connected to its appropriate game.
- Scenario:
    - The player finishes the game, and their score is calculated.
        - If the player wins, the appropriate points will be applied, accounting for any multipliers
        - If the player loses, the appropriate points will be subtracted.
    - The game system sends the player's score, player ID, and other relevant data to the leaderboard system.
    - The leaderboard system compares its score to the other players and adjusts the placements accordingly.
        - If the requirements are met, the player will either rank up or rank down in addition to their general placement.
    - The system updates the leaderboard with the player's new rank and score.

        - The leaderboard system sends confirmation to the game system that the score has been successfully recorded.
        - The updated leaderboard is displayed, showing the player's new ranking.
- Postconditions:
    - The leaderboard now reflects the current rankings including the player's accurate position.
- Exceptions:
    - If there is a network failure during score submission, the result of the win/loss does not go through.
    - If the score or the game session is for some reason invalid or corrupted, the result of the win/loss does not go through. May prompt for the player to play another round.

Process Game Result

- Primary Actor(s): player
- Goal in context: Upon completion of a game, the result of the game is processed and player's stats change accordingly
- Trigger: The game session ends with a clear winner between the player and a matched opponent.
- Preconditions: The player has successfully completed a game session.
- Scenario:
    - The player finishes the game
    - If the player won the game, then their "wins"stat for that game is incremented by 1, otherwise their "losses" stat for that game is incremented by 1
    - Their MMR for that game is adjusted accordingly
- Postconditions:
    - Either the player's wins or losses for the game that has been completed is altered
    - Their MMR is also altered
- Exceptions:
    - If the game played was a placement match, depending on which placement game it was, the MMR may not be altered
    - If the score or the game session is for some reason invalid or corrupted, the result of the win/loss does not go through. May prompt for the player to play another round.

Authentication & Profile Use Case Descriptions:

player Registration

- Primary Actor(s): New player
- Goal in Context: To create a new player account in the system to utilize customized features and monitor rankings and game history.
- Trigger: player selects the “Create an account” option in the software.
- Preconditions: player cannot be registered previously.
- Scenario:
    - player navigates to the registration page.
    - player enters required information, such as playername, email, and password.
    - System validates the provided information (format and uniqueness).
    - System creates a new player account.
    - player receives a confirmation email.
- Postconditions: After the player account is created and validated, access to service and login are made possible.
- Exceptions:
    - player attempts to register with an email that is already in use.
    - player enters invalid credentials.
- Priority: High
- When available: Iteration 1
- Frequency of use: Once per player
- Channel to actor: Click the "Create an account" option
- Channels to secondary actors: N/A
- Open issues: N/A

player Login

- Primary Actor(s): Registered player
- Goal in Context: To authenticate into the system to gain personalized access to play and view game history and rankings.
- Trigger: player selects the “Login” option.
- Preconditions: player already has an account registered.
- Scenario:
    - player navigates to the login page.
    - player enters their credentials (playername and password).
    - System verifies the credentials against the database.

    - System grants access to the player account.
- Postconditions: player is logged into their personalized system and can access their personalized dashboard and other features and can start different games being offered.
- Exceptions:
    - player enters an incorrect playername or password.
    - player account is locked due to multiple failed login attempts.
- Priority: High
- When available: Within first iteration
- Frequency of use: Often
- Channel to actor: Enter the ID and password, then click the login button
- Channels to secondary actors: N/A
- Open issues: N/A

Password Management

- Primary Actor(s): Registered player
- Goal in Context: To enable players to change their passwords for security reasons, if the player has forgotten the password or personal motives.
- Trigger: player forgot the password or wants to change password.
- Preconditions: player must have an existing account with a registered email address before they can reset.
- Scenario 1: Forgot Password
    - player navigates to the login page and clicks on the “Forgot Password” button or link.
    - player is prompted to enter their registered email address associated with the account.
    - System validates the email address against existing records in the database.
    - System sends a password reset link to the player's registered email address.
    - player clicks on the reset link in the email, which redirects to a new password reset page.
    - player enters a new password and clicks save.
    - System validates the new password and saves the updated password back into the database.
    - player receives a confirmation email regarding that password has been changed with your account.
    - player then redirected to the login page to access the system account with the new password.
- Scenario 2: Change Password
    - Log in using the player ID and password
    - player goes to account settings.
    - player navigates to password management in the security section.
    - player selects the change password button.
    - player is prompted to enter their old password and the new password they want.

    - If their current password was correct then System validates the new password and saves the updated password back into the database player will receive a message saying "Success Password Changed" player receives a confirmation email regarding that password has been changed with your account. player can exit the account setting.
- Postconditions: player's password gets updated in the system, and the player can log back into the account with the new password.
- Exceptions:
    - player enters an email address that is not registered in the system.
    - System fails to send the email due to a technical or network issue.
    - player entered a password that does not meet the password requirements.
- Priority: Medium
- When available: Within first iteration
- Frequency of use: Rare
- Channel to actor: player selects the "Forgot Password" link/button on the login page or selects change password button in password management.
- Channels to secondary actors: N/A
- Open issues: N/A

Edit Profile

- Primary Actor(s): Registered player
- Goal in Context: Allow players to personalize their profiles.
- Trigger: player wants to change their profile.
- Preconditions: player must have a registered account.
- Scenario:
    - player logs in to their account and goes to account settings.
    - player navigates to the profile page.
    - player selects "edit profile".
    - player is able to edit their display name, profile picture, bio and whether they want to display game stats.
    - Once player is finished they will click "Save Changes"
    - System validates the profile and saves the updated profile back into the database.
    - player can exit the account setting.
- Postconditions: Updated profile is displayed correctly.
- Exceptions:
    - System fails to save changes due to a technical issue.
    - player profile information has changed, but other players can still see the old information.
- Priority: Medium
- When available: Iteration 1
- Frequency of use: Often
- Channel to actor: player goes to settings and selects "edit profile".

- Channels to secondary actors: N/A
- Open issues: N/A

View Personal Profile

- Primary Actor(s): Logged-in player
- Goal in Context: To view personal profile information.
- Trigger: player wants to check their personal profile details.
- Preconditions: player must be logged in.
- Scenario:
    - player navigates to their profile page.
    - player select the option to "View Public Profile"
- Postconditions: player's profile information is viewable.
- Exceptions:
    - player submits invalid data.
    - System fails to show the profile due to a technical issue.
- Priority: Low
- When available: Within first iteration
- Frequency of use: Rare
- Channel to actor: player selects "View Profile" option
- Channels to secondary actors: N/A
- Open issues: N/A

Search Profile

- Primary Actor(s): Logged-in player
- Goal in Context: player desires to search another player's profile.
- Trigger: player selects the "Search profile" option in the profile.
- Preconditions: player must be logged-in.
- Scenario:
    - player navigates to the profile page.
    - player selects the option to "Search profile".
    - player enters playername to search for another player.
    - The player's profile corresponding to playername will be shown.
- Postconditions: player's profile is shown.
- Exceptions:
    - Non-existent playername is entered.
    - The system fails to retrieve the player's profile.
- Priority: Medium
- When available: Within first iteration
- Frequency of use: Often
- Channel to actor: player navigates to "Search Profile" option in profile page.
- Channels to secondary actors: N/A

- Open issues: N/A

View Game History

- Primary Actor(s): Logged-in player
- Goal in Context: To view the total games played, total win and loss.
- Trigger: player wants to see their game history.
- Preconditions: player must be logged into their account and have played a game in the past.
- Scenario:
    - player navigates to the game history section under profile.
    - System will display a list/record of games played in the past with details that include date, opponents, scores, and outcomes.
- Postconditions: player can now review their game history.
- Exceptions:
    - player has no record of game history.
    - System fails to retrieve game history due to a technical or network traffic issue.
- Priority: High
- When available: Within first iteration
- Frequency of use: Often
- Channel to actor: player clicks “Game History” option in the Profile.
- Channels to secondary actors: N/A
- Open issues: N/A

Update Game Ranking

- Primary Actor(s): System
- Goal in Context: Update player rankings based on game played and their outcome (win/loss).
- Trigger: Completion of a game session.
- Preconditions: Game must be completed, and results finalized.
- Scenario:
    - Finishes the Game session and results are being recorded.
    - System will calculate the new rankings based on the latest results.
    - System updates the rankings in player profiles accordingly.
- Postconditions: player rankings are updated to reflect recent game outcomes.
- Exceptions:
    - System fails to calculate or update rankings due to a technical error.
    - After the game is finished, the information is incorrectly updated.
- Priority: High
- When available: Within first iteration
- Frequency of use: Often
- Channel to actor: The game is finished and the information is updated.

- Channels to secondary actors: N/A
- Open issues: N/A

Delete Account

- Primary Actor(s): player
- Goal in Context: player desires to delete their account.
- Trigger: player selects the “Permanently Delete Account” option in the account settings.
- Preconditions: player must currently have an account.
- Scenario:
    - player logs into their account.
    - player clicks on account settings.
    - player navigates to the account management section.
    - player clicks on "Permanently Delete Account"
    - player is prompted to confirm by entering their password.
    - player receives a confirmation email about their account being deleted.
- Postconditions: players account has been deleted.
- Exceptions:
    - System fails to delete the account.
    - Account trying to be deleted did not exist.
- Priority: High
- When available: Within first iteration
- Frequency of use: Rare
- Channel to actor: player navigates to the "Permanently Delete Account” option in their account settings.
- Channels to secondary actors: N/A
- Open issues: N/A

Missing use cases added in by the GUI team

Verify account

● Primary Actor(s): Player 1

● Secondary Actor: External Authenticator

● Goal in context: Player confirms their account (through email verification)

and is granted full access to the OMG platform

● Trigger: Player registers email address on the platform

● Preconditions:

- ○ Player has not previously registered
- ○ Player has eligible email address to verify their authenticity

- Scenario:
    - Player chooses to ‘register’ from the main page
    - Player enters personal information including their email address
    - If the email is legitimate, a verification message will be sent to the entered email address from the external authenticator
    - Player verifies the email address by clicking a link in the email
- Postconditions:
    - Email is added to the registered accounts list
    - Player is logged in, and is granted full access to the platform
- Exceptions:
    - Player does not have an eligible email address
    - Authentication issues (due to inability to open links, etc.)
- Priority: High
- When available: Within first iteration
- Frequency of use: Often
- Channel to actor: via clicking of mouse on the GUI screen
- Channel to secondary actors: N/A
- Open issues: N/A

Manage account

- Primary Actor(s): Player 1
- Secondary Actor: Server
- Goal in context: Player wishes to view and/or update their account details, these include notifications, privacy settings, language preferences, and security settings (password and email changes)
- Trigger: Player chooses to ‘manage account’ on the profile menu
- Preconditions:
    - Player is logged in to their account
- Scenario:
    - Player selects sub menu depending on what they wish to do with their account

    - Player enters new information; changes preference/settings
    - Prompt to confirm the changes
    - New account information will be saved on the database
- Postconditions:
    - Player's account information is up to date based on their desired changes
- Exceptions:
    - Newly entered information is not valid
    - Fail to save changes due to server issues
- Priority: Low
- When available: Within first iteration
- Frequency of use: Rarely
- Channel to actor: via clicking of mouse on the GUI screen
- Channel to secondary actors: via clicking of mouse on the GUI screen
- Open issues: N/A

Add friend

- Primary Actor(s): Player 1
- Secondary Actor: Player 2
- Goal in context: Player 1 searches for Player 2's player ID and successfully adds them to their friend's list
- Trigger: Player selects 'add friend' on the main page
- Preconditions:
    - Player 1 and Player 2 are both online (logged in) and possess verified accounts
- Scenario:
    - Player 1 enters Player 2's player ID in search bar
    - Search result will show Player 2's public profile
    - Player 1 can add Player 2 by clicking 'add'
- Postconditions:
    - Player 2 is added as Player 1's friend, vice versa

    - Player 2 will be notified
- Exceptions:
    - Invalid player ID
    - Server issues
- Priority: High
- When available: Within first iteration
- Frequency of use: often
- Channel to actor: via clicking of mouse on the GUI screen
- Channel to secondary actors: via clicking of mouse on the GUI screen
- Open issues: N/A

Challenge friend

- Primary Actor(s): Player 1
- Secondary Actor: Player 2
- Goal in context: Player 1 wishes go against Player 2 in game of their choice
- Trigger: Player 1 either invites Player 2 to play a game through their friend list or directly from the interface inside their chosen game.
- Preconditions:
    - Player 1 and Player 2 are both online (logged in) and possess verified accounts
    - Player 1 and Player 2 are both mutually added
    - The desired game is available to play for both players
- Scenario:
    - Player 1 invites Player 2 from their friend list or directly from within the game interface
    - Player 2 accepts the game invitation
    - Both players are placed inside the same game session
- Postconditions:
    - A game session successfully exists between the two players
- Exceptions:
    - Player 1 currently has no friends in their friends list
    - Player declines Player 1's invitation
    - Server issues
- Priority: Medium
- When available: Within first iteration
- Frequency of use: often
- Channel to actor: via clicking of mouse on the GUI screen
- Channel to secondary actors: via clicking of mouse on the GUI screen
- Open issues: N/A

Choose a game

- Primary Actor(s): Player 1
- Goal in context: The player picks out a game they wish to play from among OMG's catalogue
- Trigger: The player searches up or presses on the desired game's icon located on our platform
- Preconditions:
    - Player is logged in and possesses a verified account
    - Player is connected to wifi
- Scenario:
    - Player browses through the game catalogue and selects a game they wish to play
    - The game launches
- Postconditions:
    - The game the player selects launches and redirects them to the game's main menu
- Exceptions:
    - Game fails to launch due to server issues, device compatibility issues etc.
- Priority: High
- When available: Within first iteration
- Frequency of use: Often
- Channel to actor: via clicking of mouse on the GUI screen
- Channel to secondary actors: via clicking of mouse on the GUI screen
- Open issues: N/A

Concede

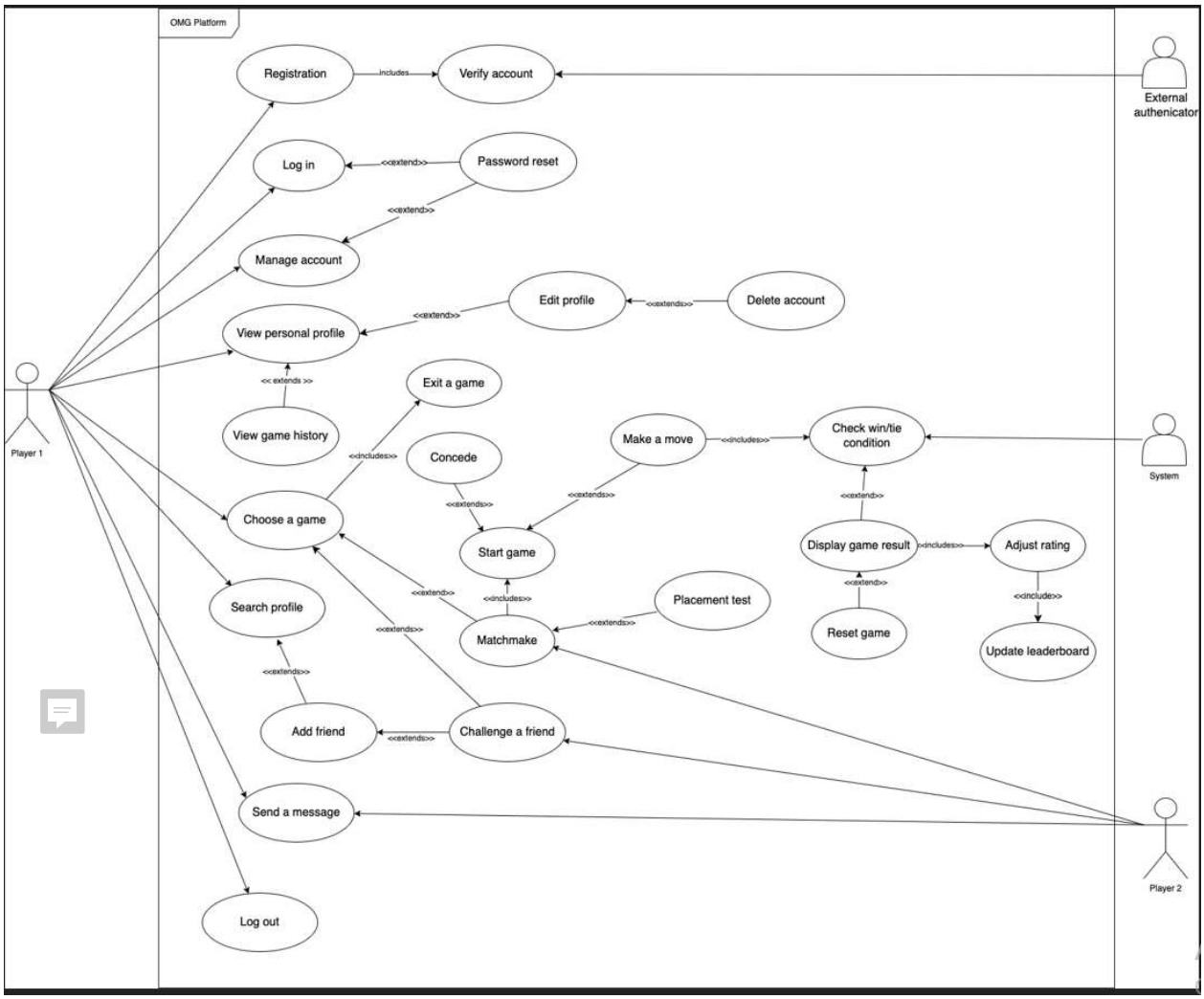
- Primary Actor(s): Player 1 and Player 2
- Goal in context: A player in the chosen game wishes to forfeit the match
- Trigger: A player chooses the "forfeit" button on the interface, surrendering the game.
- Preconditions:
    - A match is in progress
    - Both players are connected to the game session
- Scenario:
    - A player in the game session wishes to forfeit the match.
    - The forfeit button on the interface is pressed.
    - The system displays a prompt asking the player to confirm they wish to concede the match.
    - The conceding player confirms the prompt

    - The system ends the match, updating the game status, adjusting player ratings, and updating the leaderboard.
    - The system displays game results
    - Both players may return to the main menu
- Postconditions:
    - Player ratings are adjusted accordingly
    - Leaderboard is adjusted accordingly
- Exceptions:
    - Player decides against conceding
    - Server issues
- Priority: Medium
- When available: Within first iteration
- Frequency of use: often
- Channel to actor: via clicking of mouse on the GUI screen
- Channel to secondary actors: N/A
- Open issues: N/A

Log out

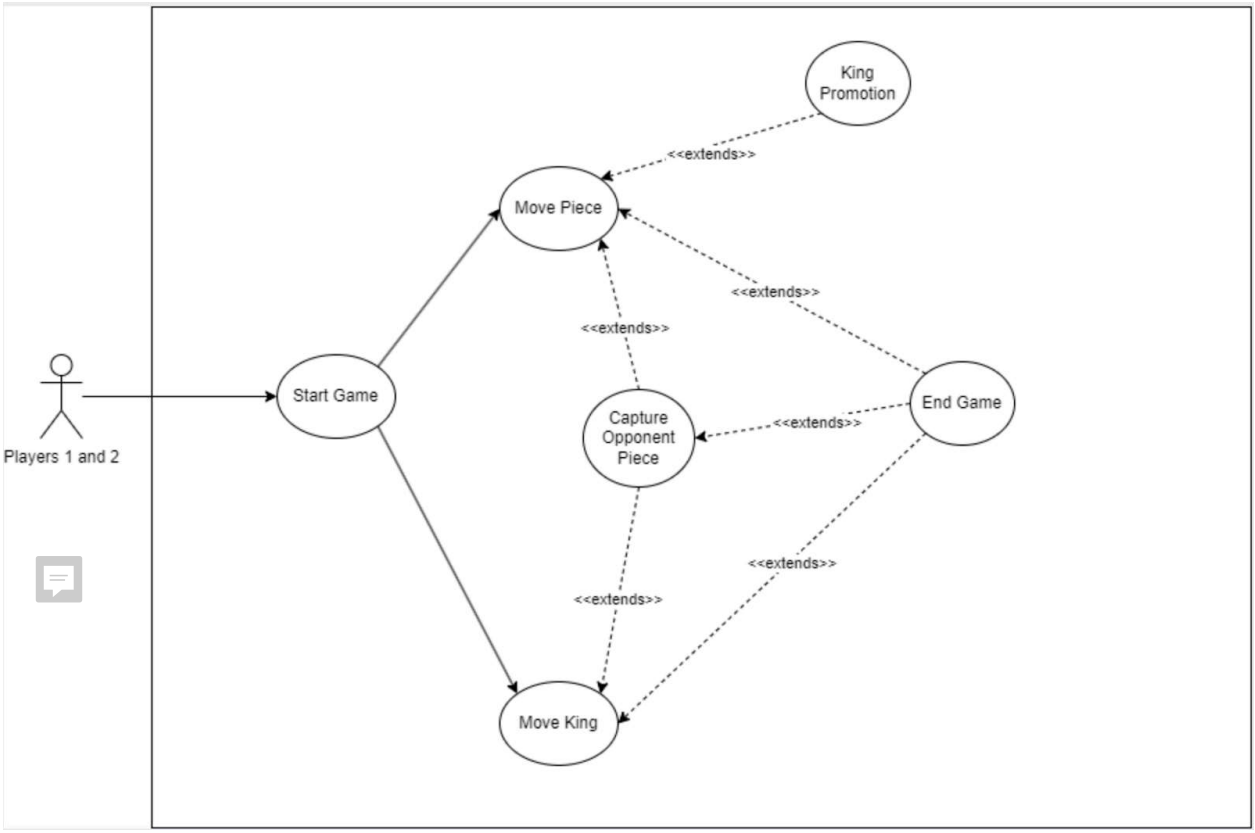
- Primary Actor(s): Player 1
- Goal in context: Player wishes to log out of the game platform
- Trigger: Player clicks on “Log out” button on the interface
- Preconditions:
    - Player is currently logged in with a verified account on the OMG platform
- Scenario:
    - The player presses on the “Log out” button located in the top right corner of the page
    - The player is redirected to the Log in page
    - A message is displayed at the top of the screen confirming that the player has been successfully logged out
- Postconditions:
    - The session is securely closed
    - Personal information and session data removed from memory
    - Player no longer has access to platform functionalities
- Exceptions:
    - Login request fails due to server issues, network issues etc.
- Priority: High
- When available: Within first iteration
- Frequency of use: Often
- Channel to actor: via clicking of mouse on the GUI screen
- Channel to secondary actors: via clicking of mouse on the GUI screen
- Open issues: N/A

GUI general use case diagram

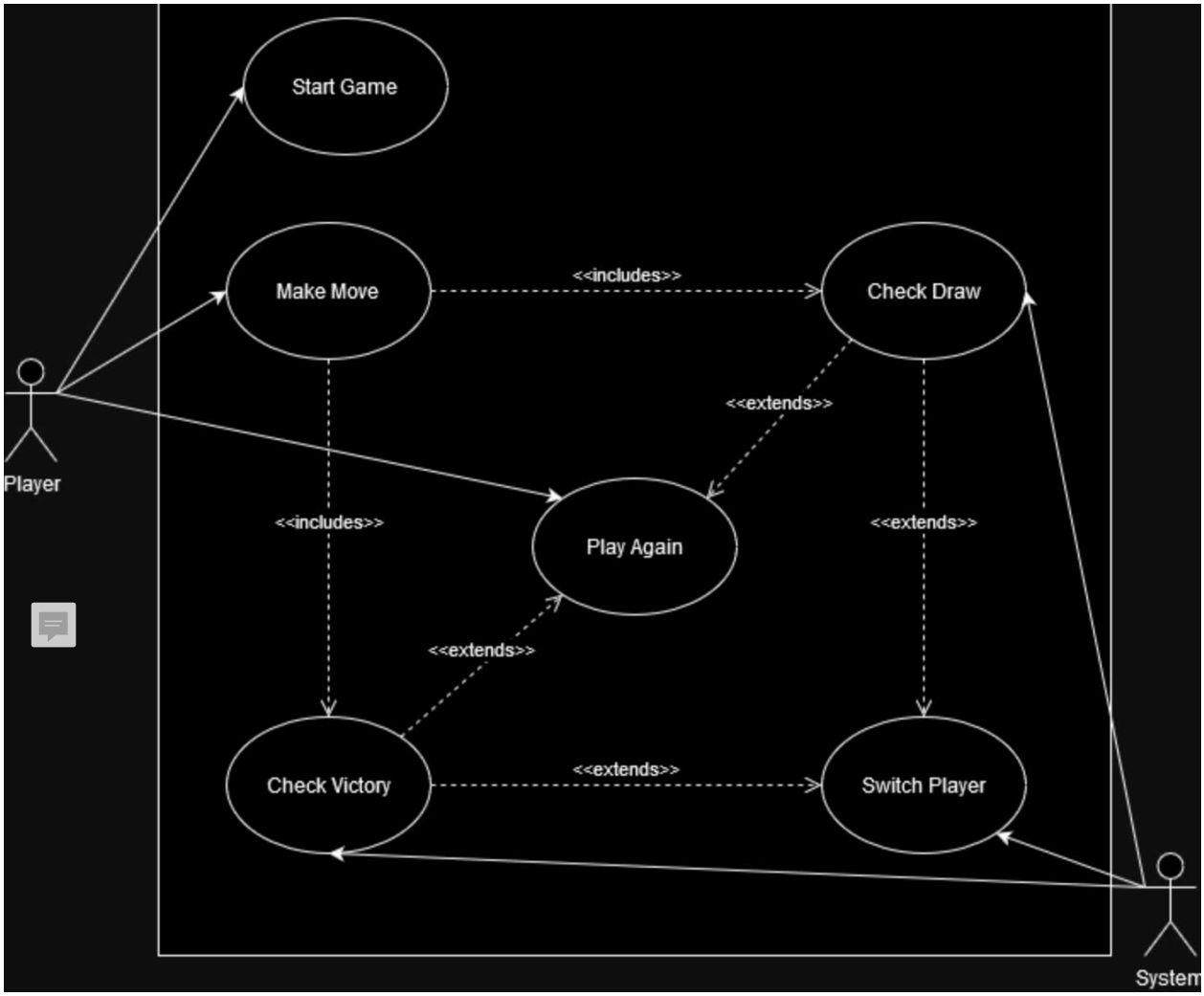


GAME LOGIC USE CASE DIAGRAMS:

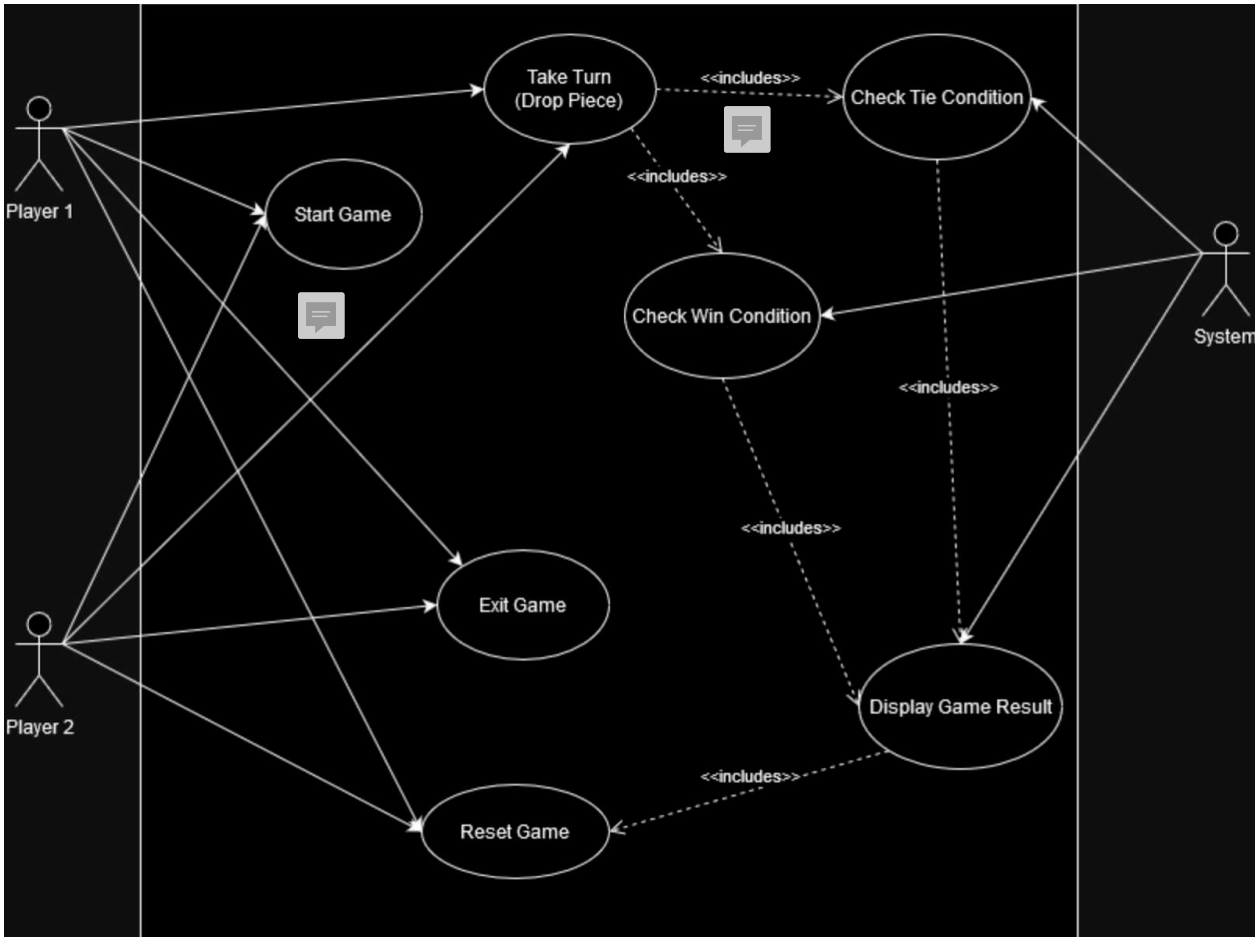
Checkers:



Tic-Tac-Toe:


Start Game
Make Move
<<includes>>
Check Draw
Player
<<extends>>
<<includes>>
Play Again
<<extends>>
<<extends>>
Check Victory
<<extends>>
Switch Player
System

Connect 4:


Take Turn
(Drop Piece)
<<includes>>
Check Tie Condition
Player 1
Start Game
<<includes>>
Check Win Condition
System
<<includes>>
<<includes>>
Exit Game
Player 2
Display Game Result
<<includes>>
Reset Game

Leaderboard and Matchmaking:

